

# Programmation objet en PHP

partie 1 : classes et objets

# PHP et les classes/les objets

- PHP est un langage incluant des fonctionnalités orientées objet : on peut programmer avec des classes, des objets, des méthodes, de la composition, de l'héritage, mais
- Un programme php :
  - exécutable en ligne de commande : `php monprog.php`
  - Accessible via une url : <http://mon.si.te/monprog.php>
- Est forcément un programme **non** objet

# un programme typique

- crée des objets et appelle des méthodes mais ...
- n'est pas lui même défini comme une méthode d'un objet ou d'une classe

```
<?php  
require_once 'vendor/autoload.php';  
$app = new Dispatcher();  
$app->run();
```

# Définition des classes

```
class User {  
  
    private int $id = 1;  
    public string $name;  
    protected ?string $email;  
  
    public function __construct(  
        int $id,  
        string $name,  
        ?string $email=null) {  
  
        $this->id = $id;  
        $this->name = $name;  
        $this->email = $email;  
    }  
  
    public function toHtml(): string {  
        return "<h2>{$this->name}</h2>" .  
            "<h3>{$this->email}</h3>";  
    }  
}
```

déclaration des attributs

constructeur

méthode

accès dans l'objet courant

# remarques

- visibilité des attributs/méthodes :
  - public / private / protected
- déclaration des attributs : le typage n'est pas obligatoire (mais obligatoire quand même !)
- méthodes : toute fonction déclarée au sein d'une classe est une méthode d'instance de cette classe
- Constructeur : un seul constructeur nommé obligatoirement `__construct` sans type de retour
- `$this` obligatoire pour référencer un attribut/méthode
  - pourquoi ? des idées ?

# utiliser des objets, appeler des méthodes

```
<?php
```

```
$p = new User(42, 'néplín', 'jean.nep@gmail.com');  
print $p->toHtml();  
$nom = $p->name;
```

# Propriétés en lecture seule

- Certaines propriétés peuvent être déclarées readonly : elles ne sont plus modifiables après initialisation
- La classe peut être readonly : toutes les propriétés sont non modifiables

```
class User {  
  
    private readonly int $id;  
    public string $name;  
  
    public function __construct(  
        int $id,  
        string $n )  
    {  
        $this->id = $id;  
        $this->name = $n;  
    }  
}  
  
// $id n'est plus modifiable après  
// construction
```

```
readonly class User {  
  
    private int $id;  
    public string $name;  
  
    public function __construct(  
        int $id,  
        string $n )  
    {  
        $this->id = $id;  
        $this->name = $n;  
    }  
}  
  
// aucune propriété modifiables après  
// construction
```

# Promotion des paramètres du constructeur

## ■ Les paramètres du constructeur

- peuvent être promus pour devenir une propriété de l'objet.
- Ces propriétés n'ont pas à être déclarées, leur initialisation par le constructeur est implicite
- Tout paramètre précédé d'un modificateur (private/public/protected/readonly) est promu

```
<?php
class User {
    public ?string $email=null;
    public function __construct( private int $id,
                                public string $name) {
    }
}
$p = new User(42, 'néplin', );
$p->email = 'jean.neplin@gmail.com';
$nom = $p->name;
```



```
interface Vehicule {  
    public function accelere( int $delta): int ;  
    public function freine(int $delta) : int ;  
}
```

# interfaces

```
class Velo implements Vehicule {  
    protected int $vitesse=0;  
  
    public function __construct(  
        protected readonly string $couleur) { }  
  
    public function accelere(int $delta): int {  
        $this->vitesse += $delta;  
        return $this->vitesse;  
    }  
    public function freine(int $delta): int {  
        $this->vitesse += $delta;  
        return $this->vitesse;  
    }  
}
```

# Héritage et surcharge

```
class VeloElectrique extends Velo {  
    protected int $energieBatterie = 400;  
    protected int $positionMoteur = 1 ;  
    protected int $autonomie = 50;  
  
    public function __construct(string $c, int $eb,  
                                int $pm, int $a) {  
        parent::__construct($c);  
        $this->energieBatterie = $eb;  
        $this->positionMoteur = $pm;  
        $this->autonomie = $a;  
    }  
    public function accelere(int $delta): int {  
        $this->autonomie = $this->autonomie - $delta;  
        return parent::accelere($delta);  
    }  
}
```

accès à la méthode parente surchargée

- Une méthode ou une classe qualifiée avec `final` ne peut pas être surchargée dans/par une sous classe

```
class Velo implements Vehicule {  
    protected int $vitesse=0;  
  
    public function __construct(  
        protected readonly string $couleur) { }  
  
    public function accelere(int $delta): int { ... }  
    public final function freine(int $delta): int { ... }  
}
```

```
final class VeloElectrique extends Velo {  
    protected int $energieBatterie = 400;  
    protected int $positionMoteur = 1 ;  
    protected int $autonomie = 50;  
  
    public function __construct(string $c, int $eb,  
                                int $pm, int $a) { ... }  
    public function accelere(int $delta): int { ... }  
}
```

# constantes

- On peut définir des constantes dans les classes et les interfaces ; les constantes sont `public` par défaut

[illegible]

# variables et méthodes statiques

- déclarée avec le mot-clé `static`
- utilisation :
  - `<nomClasse>::$variable`
  - `<nomClasse>::methode()`
  - `$className::methode()` | `$className::`  
`$variable`
  - `self::$variable` | `self::methode()` : `self`  
référence la classe de définition
  - `static::$variable` | `static::methode()` :  
`static` référence la classe d'appel

```

class A {
    public static $nom = "marcel\n";
    public static function qui() {
        print self::$nom;
    }
    public static function run() {
        self::qui();
        static::qui();
    }
}

```

```

$classeA = "A";
print A::$nom;
print $classeA::$nom;
A::qui();
A::run();

```

marcel  
marcel  
marcel  
marcel  
marcel

```

class B extends A {
    public static $prenom = "mireille\n";
    public static function qui() {
        print self::$prenom;
    }
}

```

```

B::run();

```

marcel  
mireille

# conventions de code : PSR-1

*php standard recommendation - 1*

*A respecter impérativement et scrupuleusement*

- Voir <http://www.php-fig.org/psr/psr-1/>
- utiliser <?php
- utf8
- noms de classes et interfaces : **StudlyCaps**
- méthodes : **camelCase()**
- attributs, variables : **\$camelCase**, **\$StudlyCaps**, **\$under\_score** à condition d'être homogène dans le projet

- un fichier déclare une classe ou des fonctions, **OU** produit des résultats, jamais les 2
- Les fichiers contenant des déclaration de classe sont nommés avec le nom de la classe

```
<?php
require_once 'Vehicule.php';
require_once 'Velo.php';
require_once 'VeloElectrique.php';

$vae = new VeloElectrique('rouge', 500,
                           VeloElectrique::MOTEUR_CENTRAL, 75);

$vae->accelere(55);
$vae->freine(5); $vae->freine(10);
$vae->freine(30);
print "trop tard";
```

```
<?php

class Velo implements Vehicule {...}

$monVelo = new Velo('noir');
```